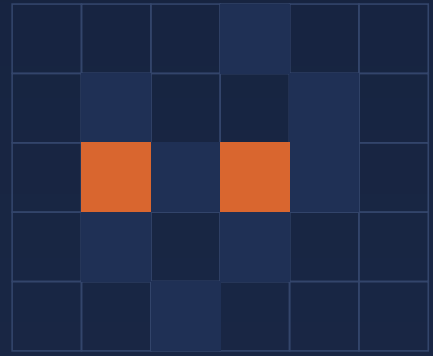


SPRING BOOT · JAVA 21 · MODULAR MONOLITH

PostForge



커뮤니티 게시판 위에 신상품 출시 뉴스 자동 게시와
구매 판단 투표를 엮은 백엔드 — 설계 근거까지 담은 상세 문서

11개 Gradle 모듈의 허용 의존성을 명시하고 역방향 결합을 core 포트로 끊었다.
인증 · 조회수 · 아웃박스 · 외부 연동에는 실패 시 동작(fail-closed / degraded)과
트레이드오프를 함께 기록했다. 이 문서는 그 구조와 판단 근거를 정리한 것이다.

CONTENTS

01 프로젝트 개요	06 수집 파이프라인과 출시 뉴스	11 성능 엔지니어링
02 기술 스택과 실행 환경	07 AI와 RAG	12 설계 결정 기록 (ADR)
03 아키텍처 — 모듈러 모놀리스	08 트랜잭셔널 아웃박스	13 트레이드오프와 한계
04 인증과 인가	09 데이터 설계	14 운영과 배포
05 게시판 도메인	10 API 표면	15 문서 체계와 마무리

11
모듈

22
소유 테이블

53
REST 엔드포인트

146
테스트 클래스

REPOSITORY
postforge · dev.iamrat

BRANCH
release/postforge

COMMIT
412f3d4c

PUBLISHED
2026-07-30

목차

PostForge의 구조와 그렇게 만든 이유를 15개 장으로 나눠 정리했다.

읽는 순서 제안

5분 — 1장 개요와 3장 아키텍처의 모듈 그래프. 30분 — 여기에 6장 출시 뉴스 파이프라인과 11장 성능. 깊이 있게 — 12장 설계 결정과 13장 트레이드오프까지.

01. 프로젝트 개요	4
핵심 기능	4
이 프로젝트가 증명하려는 것	4
02. 기술 스택과 실행 환경	5
배포 아키텍처	5
로컬 실행	6
03. 아키텍처 — 모듈러 모놀리스	7
허용 의존성 그래프	7
경계를 테스트로 강제한다	7
모듈 책임과 허용 의존성	7
core와 support를 나눈 이유	8
모듈 내부의 계층	8
04. 인증과 인가 — auth	10
핵심 규칙	10
오류가 어디서 응답되는가	10
05. 게시판 도메인 — board	12
목록 조회 — batch로 파생 데이터를 채운다	12
상세 조회와 조회수 버퍼	12
좋아요는 왜 Redis write-behind가 아닌가	12
구매 판단 투표	13
파일 업로드 — 앱 서버는 바이트를 중계하지 않는다	13
06. 수집 파이프라인과 출시 뉴스 자동 게시	14
상품 수집	14
출시 뉴스 자동 게시 — 결정적 게이트를 LLM 앞에 둔다	14
게시 결과에 남는 것	15
문서 적재 (RAG 입력)	15

07. AI와 RAG — ai	16
입력 방어와 출력 방어	16
게시글 초안 생성	16
비용 경계	17
검증 레이어	17
08. 트랜잭셔널 아웃박스 — messaging	18
구성 요소	18
정직하게 말하면	18
09. 데이터 설계	19
소유권 규칙	19
관계형 테이블 소유권	19
Redis 키와 S3 소유권	20
Migration 규칙	20
10. API 표면	21
인증 표기와 공통 응답	21
엔드포인트	21
11. 성능 엔지니어링	23
접근 — 한 번에 한 변수만	23
① 시나리오 분리 — CPU 포화 제거	23
② N+1 제거와 batch size	23
③ 조회수 Redis 캐시 A/B — 같은 최적화가 API마다 반대로 작동했다	23
외부 연동 계측	24
12. 설계 결정 기록 (ADR)	25
ADR-001 · 조회수에 Redis 사용	25
ADR-002 · Refresh Token Rotation	25
ADR-003 · 모듈러 모놀리스 (Accepted)	25
13. 트레이드오프와 한계	27
14. 운영과 배포	29
CI 구성	29
관측	29
환경변수 그룹	29
테스트 실행	30
15. 문서 체계와 마무리	31
정리	31

SECTION 01

프로젝트 개요

01

PostForge는 Spring Boot 3 / Java 21 기반의 DDD-lite 모듈러 모놀리스 커뮤니티 백엔드다. 게시판 도메인(게시글 · 댓글 · 좋아요 · 조회수 · 파일) 위에 신상품 출시 뉴스 자동 게시, 가격 판정 기반, 구매 판단 투표를 엮었다.

이 프로젝트의 목표는 기능의 개수가 아니라 **경계**다. 11개 Gradle 모듈이 서로의 구현 대신 허용된 application API 또는 core 포트만 협력하고, 그 규칙을 ArchUnit 테스트로 강제한다. 외부 API · Redis · LLM처럼 실패할 수 있는 모든 연동에는 "실패하면 어떻게 되는가"를 fail-closed / degraded 중 하나로 명시했다. 성능 개선은 추측이 아니라 CPU · SQL count · p95를 분리해 측정 한 뒤 before/after로 남겼고, 캐시가 오히려 성능을 악화시킨 사례도 숨기지 않고 기록했다.

11

Gradle 모듈

554

Java 파일

146

테스트 클래스

22

소유 테이블

53

REST 엔드포인트

핵심 기능

영역	현재 구현
인증	JWT access token, Redis refresh token rotation, OAuth2(Google · Naver · Kakao), 이메일 인증, 로그인 abuse 보호
게시판	게시글, 댓글/대댓글(1-depth), 좋아요, 조회수 버퍼, S3 presigned URL 업로드, 작성자 소유권 검증
출시 뉴스	Naver API HUB News 후보를 결정적 gate와 AI draft port로 처리해 PRODUCT_LAUNCH_NEWS 게시글로 자동 발행
가격 판정	저장하지 않는 응답 전용 판정. Naver Shopping 검색 종료로 대체 소스 연결 전까지 503 반환
구매 판단	자동 게시된 출시 뉴스에 한해 BUYABLE / UNSURE / WAIT 투표. 계정당 1표
상품 수집	tracked keyword → collection job → raw product → catalog product → price snapshot 파이프라인
AI / RAG	Spring AI, OpenAI-compatible chat, PgVector 문서 검색과 게시글 초안 생성, 결정적 입출력 안전 가드
운영 기반	Flyway baseline, Docker layered runtime image, 구조화 로그, Micrometer / Prometheus / Grafana

이 프로젝트가 증명하려는 것

- **경계 있는 설계** — 기능 모듈이 허용된 application API 또는 core 포트만 협력하고 서로의 구현에는 결합하지 않는다.
- **근거 있는 성능 개선** — 병목을 추측이 아니라 CPU / SQL count / p95로 분리하고 before/after를 남겼다.
- **장애를 전제하인 인프라 판단** — Redis · 외부 API · LLM 호출마다 실패 시 동작을 fail-closed 또는 degraded로 명시했다.
- **정직한 한계 기록** — 알려진 레이스 컨디션, 미완성 확장 지점, 측정의 범위를 숨기지 않고 문서화했다.

SECTION 02

기술 스택과 실행 환경

02

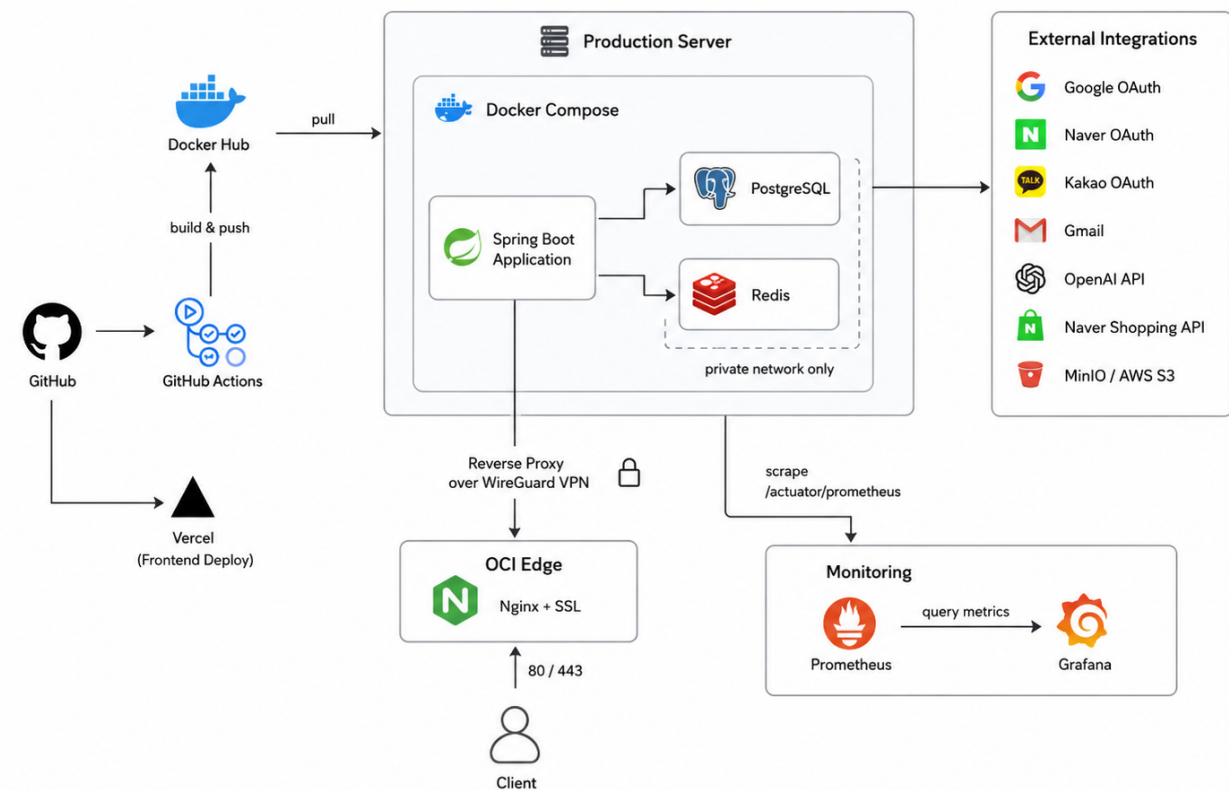
라이브러리는 "쓸 수 있어서"가 아니라 이 시스템이 실제로 만족해야 하는 요구에서 골랐다. 각 선택의 이유를 함께 적는다.

영역	기술	왜 이것인가
Runtime	Java 21 · Spring Boot 3.5.14 · Gradle 8.14.3	멀티 모듈 빌드와 toolchain 고정. record / sealed / pattern matching으로 계약 타입을 간결하게 표현
Data	PostgreSQL · PgVector · Redis · Spring Data JPA · Flyway	관계형 truth와 벡터 검색을 한 엔진에서 운영. 운영은 Flyway migration + Hibernate validate
Security	Spring Security · JWT(jjwt 0.12.7) · OAuth2 Client · Gmail SMTP	stateless access token + Redis stateful refresh의 조합을 표준 필터 체인 위에 구성
AI	Spring AI 1.0.7 · OpenAI-compatible API · PgVector	provider를 설정으로 바꿀 수 있게 port 뒤에 숨김. 로컬은 gateway를 통해 Ollama / Qwen 사용
Storage	S3-compatible storage (AWS SDK 2.x · MinIO)	앱 서버가 파일 바이트를 중계하지 않도록 presigned URL 방식만 사용
API	Spring MVC · SpringDoc OpenAPI 2.8.17	실행 중 Swagger UI가 현재 스키마의 살아 있는 정보 역할
Test	JUnit 5 · Spring Boot Test · ArchUnit 1.4.1	기능 테스트와 별개로 모듈 경계 자체를 테스트로 강제
Operations	Docker Compose · GitHub Actions · Prometheus · Grafana · ECS JSON logging	layered image로 registry cache 효율을 높이고 metric은 actuator에서 노출

배포 아키텍처

PostForge

Architecture Diagram



GitHub Actions가 테스트 통과 후 runtime image를 Docker Hub에 게시하고, 운영 서버는 Docker Compose로 이를 내려받아 실행한다. PostgreSQL과 Redis는 private network에만 노출되며, 외부 접근은 WireGuard VPN 위의 Nginx reverse proxy를 거친다.

로컬 실행

필수 도구 — Java 21+, Docker Compose

```
cp .env.example .env
docker compose -f docker-compose.local.yml up -d
./gradlew :app:bootRun
```

전체 환경변수와 로컬 기본값은 .env.example이 정본이다. 실제 secret이 든 .env는 커밋하지 않으며, bootRun이 루트 .env를 자동으로 읽는다.

로컬 LLM 연결 경로

애플리케이션은 Ollama를 직접 호출하지 않고 OpenAI-compatible gateway를 거친다. 따라서 LLM_CHAT_BASE_URL에는 Ollama 주소가 아니라 gateway 주소를 넣는다.

PostForge app → OpenAI-compatible LLM gateway → Ollama → Qwen model

SECTION 03

03

아키텍처 — 모듈러 모놀리스

하나의 Spring Boot 애플리케이션으로 배포하되, 내부는 기능 경계별 Gradle 모듈로 나눈다. 목표는 초기 운영 복잡도를 낮게 유지하면서 서비스 분리가 필요해질 때 경계를 식별하기 쉽게 만드는 것이다.

허용 의존성 그래프



허용하지 않는 역방향

기능 모듈 → app · board → auth · ingest → ai · core → 다른 모듈

A → B는 A의 코드가 B의 타입을 참조할 수 있다는 뜻이며, 반대 방향은 별도로 허용하지 않는다.

경계를 테스트로 강제한다

문서에 적힌 규칙은 시간이 지나면 어긋난다. 그래서 위 허용 모듈 그래프는 ArchUnit 기반 ModuleBoundaryTest가 검사하고, 외부 라이브러리 전이 의존성은 dependencyInsight로 확인한다. 규칙을 어기면 빌드가 깨진다.

경계만 빠르게 확인 / 의존성 변경 후 전체

```
./gradlew :app:test --tests dev.iamrat.app.architecture.ModuleBoundaryTest
./gradlew test
```

모듈 책임과 허용 의존성

모듈	직접 참조할 수 있는 모듈	책임
app	core, support, 모든 기능 모듈	실행 진입점, 전체 조립, JPA scan bootstrap, route / security / OpenAPI 정책
core	없음	공통 DTO · 예외 · principal / API metadata와 모듈 간 port 계약
support	core	Redis guard primitive, JPA auditing, request logging, MVC 예외 응답
auth	core, support	계정, 로그인, JWT, OAuth2, 이메일 인증, 인증/인가 오류 응답
board	core, support	게시글, 댓글, 좋아요, 파일/S3, 조회수, 구매 판단 투표
source	없음	외부 상품·뉴스 source adapter와 MOCK / NAVER routing contract
ingest	core, source, catalog, price	문서 적재, 상품·뉴스 수집 orchestration, tracked keyword, collection job
catalog	core	정규화 상품, offer, 카테고리, product embedding, 유사 상품 후보
price	core, source, catalog	가격 스냅샷 이력, 가격 이력 조회, 응답 전용 가격 판정
ai	core, catalog	AI 채팅, 게시글 초안, product embedding adapter, LLM / PgVector 설정
messaging	core	outbox 저장, in-process dispatch, optional relay / MQ adapter 경계

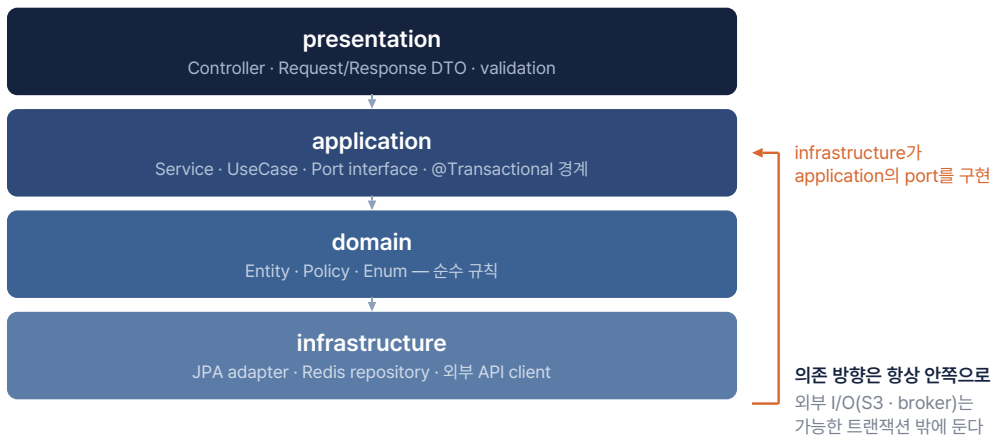
core와 support를 나눈 이유

둘 다 여러 모듈에서 쓰이지만 성격이 다르다. 하나로 합치면 "공용"이라는 이름의 쓰레기통이 되기 쉬워서, 무엇이 들어가면 안 되는 지까지 규칙으로 못 박았다.

구분	core	support
역할	모듈 간 compile-time 계약 (shared kernel)	Spring 실행에 필요한 공통 인프라 구현
들어가는 것	interface, record, 공통 DTO · 예외 · principal	@Configuration, @Bean, 공통 web / redis / persistence helper
들어가면 안 되는 것	Redis · OpenAPI · S3 · LLM · JPA 구현, feature service	도메인 계약과 규칙, feature service
참조 방식	기능 모듈이 컴파일 시 참조	app이 조립하고 필요한 기능 모듈만 참조

예를 들어 PostWriter와 SourceDocumentCommand는 모듈 간 약속이므로 core에 둔다. Redis TTL primitive와 MVC 예외 응답 변환은 공통 Spring 구현이므로 support에 둔다. 기능 모듈 내부의 auth.support.error 같은 패키지는 전역 support 모듈과 무관한 module-local namespace이며, 다른 모듈도 알아야 하는 계약은 여기에 두지 않고 core로 올린다.

모듈 내부의 계층



역방향 결함을 끊은 세 지점

- **board** → **auth** 금지 — 컨트롤러는 인증 구현이 아니라 core의 공통 principal 계약(UserPrincipal)만 참조한다. 권한과 소유권 판단은 언제나 accountId 기준이다.
- **ingest** → **ai** 금지 — 문서 적재는 Spring AI VectorStore API로, 출시 뉴스 초안은 core의 draft 생성 포트로 호출한다. 실제 PgVector bean은 ai가 만든다.
- **ingest** → **board** 금지 — 출시 뉴스 발행은 core의 PostWriter 포트를 거치고, 구현체만 board.post.application.BoardPostWriter에 둔다.

SECTION 04

04

인증과 인가 — auth

access token은 stateless JWT로, refresh token은 Redis에 저장하는 stateful credential로 나눠 다룬다. 로그아웃 · 장기 세션 · 탈취 토큰 무효화 같은 상태성 요구는 refresh 쪽에서만 처리한다.

access token

stateless JWT · Authorization header · 만료 전까지 유효

refresh token

Redis refresh_token:{accountId} · stateful · 재발급마다 rotation

- 1 POST /api/auth/token/reissue
요청 token 의 subject 에서 accountId 추출
- 2 Redis 저장값과 상수 시간 비교
불일치 → INVALID_TOKEN (탈취 token 재사용 차단)
- 3 계정 활성 상태 · role 재조회
비활성 계정은 여기서 거절
- 4 새 access + 새 refresh 발급
Redis 가운 새 token 으리 교체 (rotation)

알려진 한계 — validate + replace 가 단일 원자 연산이 아니다.
동시 재발급이 들어오면 둘 다 검증을 통과하고 마지막 저장분만 살아남는다. 해법은 Redis Lua / compare-and-set.

핵심 규칙

- 권한과 소유권 판단은 username이나 nickname이 아니라 accountId를 기준으로 한다.
- OAuth2 redirect URL에는 refresh token이나 access token을 노출하지 않는다. provider callback 이후 프론트엔드에는 짧은 TTL의 1회성 exchange code만 전달한다.
- 존재하지 않는 username과 잘못된 password는 모두 INVALID_CREDENTIALS로 응답해 계정 존재 여부를 노출하지 않는다.
- 계정 비활성 상태는 login · token reissue · OAuth2 exchange 세 지점 모두에서 거절한다.
- OAuth2 / 이메일 인증의 Redis state 장애는 fail-closed다. 상태를 잃느니 요청을 실패시킨다.
- 로그인 실패 · 이메일 발송 · OAuth2 exchange code는 Redis guard와 TTL state로 보호한다.

오류가 어디서 응답되는가

인증 오류는 발생 위치에 따라 MVC Advice가 처리할 수도, Security filter handler가 처리할 수도 있다. 이 경로를 섞으면 같은 상황에 다른 응답이 나가므로 우선순위를 고정했다.

발생 위치 / 예외	처리 클래스	응답
MVC BadCredentialsException	SecurityExceptionHandler	401 INVALID_CREDENTIALS
MVC UsernameNotFoundException	SecurityExceptionHandler	401 INVALID_CREDENTIALS
MVC DisabledException	SecurityExceptionHandler	403 ACCOUNT_NOT_ACTIVE

발생 위치 / 예외	처리 클래스	응답
MVC AccessDeniedException	SecurityExceptionHandler	403 ACCESS_DENIED
보호 경로의 미인증 요청	JwtAuthenticationEntryPoint	401 UNAUTHORIZED
인증됐지만 권한이 부족한 요청	JwtAccessDeniedHandler	403 FORBIDDEN
그 밖의 MVC 예외	support.ExceptionResponseHandler	500 INTERNAL_SERVER_ERROR

SecurityExceptionHandler는 HIGHEST_PRECEDENCE, 공통 ExceptionResponseHandler는 LOWEST_PRECEDENCE다. 인증 예외는 auth 정책이 먼저 처리하고, 매핑되지 않은 MVC 예외만 공통 handler로 넘어간다. 한편 JwtAuthenticationFilter는 유효하지 않거나 만료된 JWT를 발견해도 예외를 던지지 않고 SecurityContext를 비운 뒤 필터 체인을 계속 진행한다 — 보호 경로면 이후 EntryPoint가 401을, 공개 경로면 익명 요청으로 정상 처리된다.

route 인가는 app이 조립한다

PostForgeAuthorizationRules가 PUBLIC / PUBLIC_POST / PUBLIC_GET / USER_OR_ADMIN / ADMIN 다섯 그룹으로 경로를 분류하고, 마지막은 anyRequest().denyAll()이다. 새 엔드포인트를 추가하고 규칙에 등록하지 않으면 열리는 대신 **닫힌다** — 실수로 공개되는 방향의 사고를 구조적으로 막는다.

SECTION 05

05

게시판 도메인 — board

게시글 · 댓글 · 좋아요 · 조회수 · 파일 · 구매 판단 투표. 같은 게시판이라도 데이터의 성격에 따라 정합성 요구가 다르고, 그에 따라 저장소와 실패 정책을 다르게 선택했다.

목록 조회 — batch로 파생 데이터를 채운다

GET /api/posts는 keyword · category · boardCategory · publishOrigin 필터를 JPA Specification으로 조합해 페이지징 조회한다. 핵심은 그 다음이다. 목록 전체의 파생 데이터 — 좋아요 수와 본인 좋아요 여부, 조회수(Redis, miss는 DB fallback), 댓글 수, 투표 집계, reference link — 를 **postIds** 단위 **batch** 조회로 한 번에 채운다. 게시글 N건을 돌며 건별로 조회하면 그대로 N+1이 된다.

상세 조회와 조회수 버퍼

상세 조회는 **인증된 사용자일 때만** 조회수를 올린다. 비회원 조회는 카운트하지 않는다. 매 요청마다 posts.views를 즉시 update 하면 write amplification과 row contention이 생기므로, Redis를 write buffer로 두고 변경된 게시글만 스케줄러가 반영한다.

① 요청 경로 — GET /api/posts/{postId} (인증 사용자만)

setIfAbsent(post:viewed:{id}:{accountId}, 24h)
이미 있으면 즉시 종료 — 24시간 1회

get(post:views:{id})
miss 면 DB posts.views 로 fallback

increment(post:views:{id})
DB update 없이 Redis 에서 +1

sadd(post:views:dirty, id)
변경된 postId 만 동기화 대상

② 스케줄러 — 5분 주기 동기화

rename(dirty → dirty:processing)

smembers(dirty:processing)

각 id마다 get → UPDATE posts.views

성공한 id만 srem, 실패분은 다음 주기 재시도

rename 을 쓰는 이유

smembers → delete 사이에 들어온 dirty는 유실된다. rename은 원자적이라 이후 추가분은 새 dirty 키에 안전하게 쌓인다.
KEYS post:views:* 스캔은 O(N) 블로킹이라 운영에서 쓰지 않는다.

좋아요는 왜 Redis write-behind가 아닌가

조회수와 좋아요는 겉보기에 비슷하지만 정합성 요구가 다르다. 조회수는 잃어도 다음 요청에 DB 기준으로 복구되는 파생 데이터지만, "내가 이 글에 좋아요를 눌렀는가"는 사용자에게 직접 보이는 상태다. 그래서 좋아요의 source of truth는 post_like · comment_like 테이블이고, Redis는 보조 장치로만 쓴다.

- **guard** — LikeRequestGuard가 대상별 cooldown과 계정별 분당 30회 rate limit을 검사한다. 초과하거나 **Redis 장애면 fail-closed로 429**.

- **역등** — 동시 요청이 unique constraint `post_like(post_id, account_id)`에 걸리면 `DataIntegrityViolationException`을 무시한다. 중복 요청이 오류가 되지 않는다.
- **재계산** — `like_count`는 like row에서 언제든지 다시 계산할 수 있는 파생 컬럼이므로, 토크 후 DB에서 재집계해 갱신한다.

구매 판단 투표

투표는 자동 게시된 출시 뉴스의 신뢰 신호이므로 아무 글에나 열지 않는다. `PurchaseVoteEligibility`가 `category = PRODUCT_LAUNCH_NEWS`이면서 `publish_origin = SYSTEM_BATCH`인 게시글만 허용하고, 그 외에는 `PURCHASE_VOTE_NOT_ALLOWED`로 거절한다. 사용자 수동 작성 글과 `ADMIN_BACKFILL`은 대상이 아니다. 기존 투표가 있으면 vote type만 update하고, unique constraint (`post_id, account_id`)로 계정당 1표를 보장한다.

파일 업로드 — 앱 서버는 바이트를 중계하지 않는다

- 1 `FileTypePolicy`가 파일명과 `contentType`을 검증한다.
- 2 `post_file` metadata를 먼저 저장하고(UUID 저장명, 날짜 경로 object key) S3 presigned upload URL을 `fileId`와 함께 반환한다.
- 3 실제 바이트 업로드는 클라이언트가 S3에 직접 수행한다. 게시글 작성·수정 시 `fileId`로 연결되며, 연결되지 않은 파일은 `OrphanFileCleanupScheduler`가 정리한다.

게시글·댓글은 아직 물리 삭제다

현재 구현은 게시글/댓글을 hard delete하므로 복구나 감사 이력이 남지 않는다. MVP 범위에서 삭제 정책의 기준은 "일반 사용자에게 더 이상 노출하지 않는 것"이고, 계정은 이미 상태 기반 soft delete를 target으로 잡아뒀다. 확장 시 게시글/댓글도 상태 기반 soft delete로 전환하고 물리 삭제는 별도 보존 기간과 정리 작업으로 옮긴다. 삭제 시에는 파일 연결 해제 → Redis 조회수 캐시 삭제 → row 삭제 → `PostDeletedEvent` 기록 순서를 지킨다.

SECTION 06

06

수집 파이프라인과 출시 뉴스 자동 게시

ingest는 orchestration만 소유한다. 외부 API client는 source, 상품 truth는 catalog, 가격 이력은 price, 게시글 쓰기는 core의 PostWriter port, 벡터 저장은 Spring AI VectorStore API로 위임한다.

상품 수집

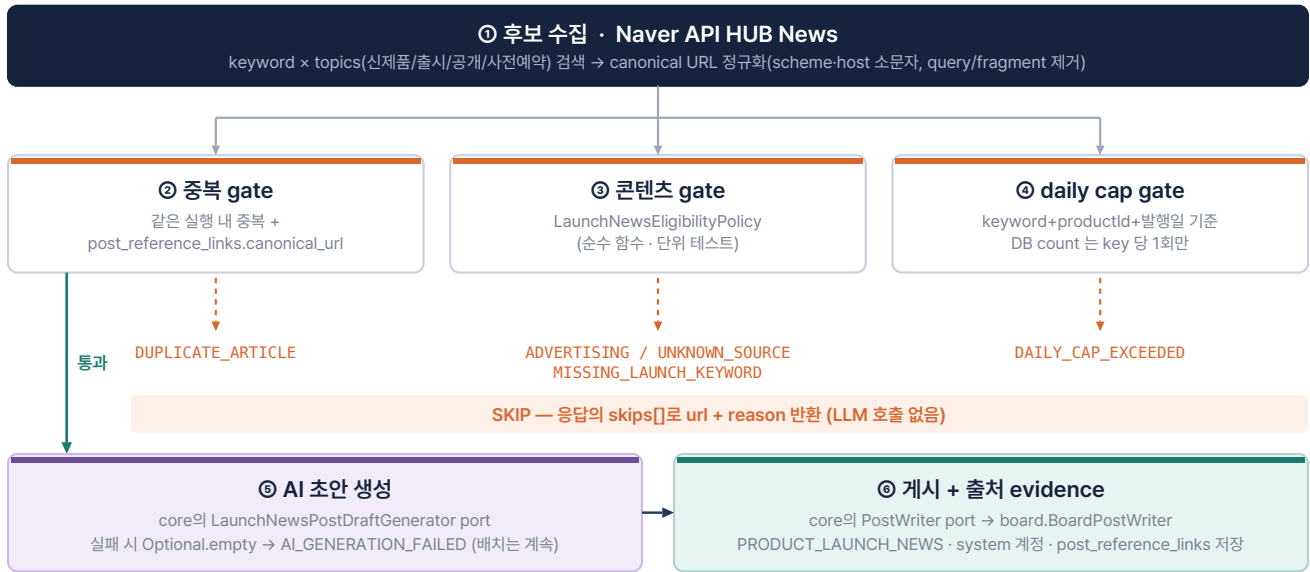
- 1 CollectProductsUseCase.collect가 collection_jobs에 PENDING row를 만들고 RUNNING으로 전이한다.
- 2 source의 SourceRequestExecutor로 외부 상품 검색(MOCK 또는 NAVER). 호출 시간은 external_source_fetch metric으로 계측한다.
- 3 item마다 raw_products에 원본 payload 저장 → catalog의 upsertWithOffer(정규화-matching) → price의 recordSnapshot.
- 4 성공 시 SUCCESS와 수집 count, 실패 시 FAILED를 기록한다. 상세 실패 원인은 내부 로그와 저장 데이터로만 추적하고 API 응답에는 노출하지 않는다.

왜 원본 payload를 그대로 보관하는가

외부 API 장애가 반복될 때 "언제부터, 어떤 keyword가, 왜" 실패했는지를 DB만으로 추적하기 위해 실패를 job row에 남긴다. 그리고 raw payload를 보관하므로 정규화 로직이 바뀌어도 외부 API를 다시 호출하지 않고 재처리할 수 있다. 스케줄 실행은 ingest.product.scheduler.enabled=true일 때만 활성화되며(@ConditionalOnProperty), cron 기본값은 매시 정각이다.

출시 뉴스 자동 게시 — 결정적 게이트를 LLM 앞에 둔다

"수집된 모든 item을 AI에 넣어 공개 게시"는 비용과 품질 양쪽에서 위험하다. 그래서 순수 함수로 검증 가능한 결정적 gate를 먼저 통과시키고, 살아남은 후보에만 LLM 비용을 쓴다.



핵심 — 결정적(deterministic) 게이트를 LLM 호출 앞에 배치한다.

통과한 후보에만 LLM 비용을 쓰고, 게시된 글에는 반드시 출처 evidence가 남아 중복-daily cap 판정의 기준이 된다.

공개 read path(목록/상세)에서는 AI를 호출하지 않는다.

Skip Reason	언제 발생하는가
DUPLICATE_ARTICLE	같은 실행 안의 중복이거나 post_reference_links.canonical_url에 이미 존재하는 기사
ADVERTISING	광고성 문구가 감지된 후보
UNKNOWN_SOURCE	출처를 신뢰할 수 없는 후보
MISSING_LAUNCH_KEYWORD	출시·예약판매·공식 발표 같은 필수 키워드가 없는 일반 기사
DAILY_CAP_EXCEEDED	keyword + productId + 발행일 기준 게시 수가 dailyCap을 초과
AI_GENERATION_FAILED	초안 생성 port가 Optional.empty를 반환 — 배치 전체를 중단시키지 않는다

게시 결과에 남는 것

필터를 통과한 후보는 PRODUCT_LAUNCH_NEWS 게시글이 된다. 작성자는 시스템 계정(accountId=0, nickname PostForge News Bot)이고, board category는 keyword · 제목 · 초안 텍스트의 규칙 매칭으로 결정한다(DIGITAL / APPLIANCE / ... 기본 GENERAL). 이어서 post_reference_links에 출처 evidence를 저장한다 — 이 레코드가 이후 중복 판정과 daily cap 계산의 기준이 되므로 게시된 글에는 반드시 출처가 남는다.

알려진 한계 — 배치 전체가 한 트랜잭션

뉴스 검색과 LLM 초안 생성이 @Transactional 안에서 실행되므로, 외부 API가 느리면 DB 커넥션 점유 시간이 길어지고 배치 전체가 하나의 트랜잭션이 된다. 후보별 트랜잭션 분리가 후속 과제다.

문서 적재 (RAG 입력)

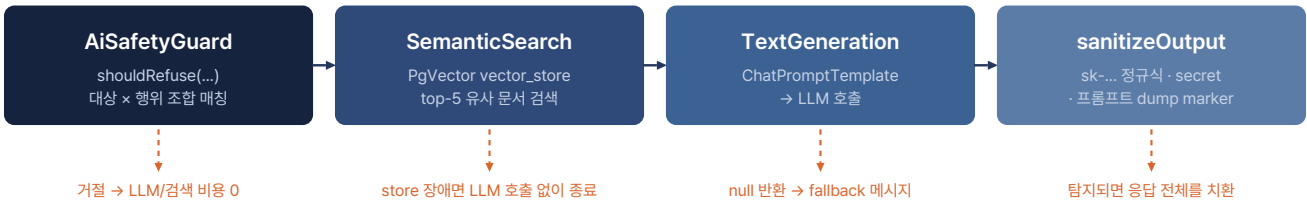
POST /api/ingest/documents는 Spring AI TokenTextSplitter로 원본 문서를 token chunk로 나눈 뒤 각 chunk의 embedding을 vector_store에 저장한다. 저장에 실패하면 원본을 보존하는 fallback이 없으므로 503을 반환한다.

SECTION 07

07

AI와 RAG — ai

LLM 호출은 application port 뒤에 두고 provider와 model은 실행 설정으로 선택한다. 모델의 판단에 의존하지 않는 결정적 규칙으로 호출 앞뒤를 감싸고, 호출 시간과 token은 metric으로 남긴다.



설계 원리

- LLM adapter는 예외를 던지지 않고 null 을 반환한다 (로그 + degraded metric). LLM 장애가 5xx 로 전파되지 않는다.
- 검색 결과 0건과 store 장애를 SearchOutcome으로 구분해, 장애일 때는 아예 LLM을 부르지 않는다.
- 부분 마스킹 대신 응답 전체 치환 — 부분 마스킹은 문맥 유추 여지를 남기기 때문.

한계 인지 — 키워드/정규식 방어는 우회 가능하므로 완전한 방어가 아니라 **defense-in-depth** 의 한 층이다.

상위 원칙은 "프롬프트와 컨텍스트에 secret을 넣지 않는다"이며, 생성된 초안은 저장되지 않는다(사용자가 별도로 게시해야 공개된다).

입력 방어와 출력 방어

단계	무엇을 하는가	설계 의도
<code>shouldRefuse(...)</code>	"대상(내부 프롬프트 · secret · 운영 정보) × 행위(보여줘 · 무시해)"의 조합으로 판단하고, 교육적 질문 (EDUCATIONAL_CONTEXT)은 예외 처리	단일 키워드 매칭의 오탐을 줄인다. 거절이면 LLM을 호출하지 않으므로 비용이 0이다
<code>sanitizeOutput()</code>	응답에 API key 형태(sk-... 정규식), secret 할당문, 내부 프롬프트 dump marker가 보이면 응답 전체를 거절 메시지로 치환	부분 마스킹은 문맥 유추 여지를 남기므로 전체를 치환한다

게시글 초안 생성

POST /api/ai/generate는 prompt뿐 아니라 topic · title · summary · tags까지 **전부 varargs**로 `shouldRefuse`에 넣는다. 어떤 필드로 주입을 시도해도 프롬프트에 섞이기 전에 걸러진다. 생성된 본문은 `sanitize`를 거치고, null이면 사용자의 원본 prompt로 fallback한다. title/summary는 사용자가 준 값을 우선하고 없으면 본문에서 유도하며(제목 100자 · 요약 500자 절단), tags는 중복 제거 후 최대 20개다.

AI 출력은 저장되지 않는다

초안 API는 결과를 DB에 남기지 않는다. 사용자가 게시하려면 별도로 POST /api/posts를 호출해야 한다. AI 출력이 사람의 검토 없이 public 데이터가 되지 않게 하는 경계다.

비용 경계

AI 호출은 **명시적 사용자 요청**(chat / generate) 또는 **gate를 통과한 admin · system 경로**(launch-news)에서만 발생한다. 게시글 목록/상세 같은 public read path는 어떤 경우에도 AI를 호출하지 않는다. 외부 fetch · DB persist · LLM generation은 각 각 별도 timer로 분리 계측해, HTTP latency만 보고 튜닝 대상을 잘못 고르는 일을 막는다.

검증 레이어

레이어	대상	확인 내용
단위 테스트	ChatService, PostDraftGenerationService, LaunchNewsPostDraftGenerationService, AiSafetyGuard	safety pre-check, fallback, sanitize, public post 차단 정책
어댑터 테스트	LlmTextGenerationAdapter, LlmProductEmbeddingClient, PgVectorSearchAdapter	Spring AI ChatModel / EmbeddingModel / VectorStore 호출 변환과 실패 metric
HTTP slice	ChatController, PostDraftGenerationController	/api/ai/chat, /api/ai/generate 요청·응답 shape와 validation
수동 smoke	ai-chat-smoke.http, naver-source-smoke.http	로컬 gateway · 인증 API · AI endpoint · Naver source write path 연결 확인

자동 테스트는 실제 LLM 비용과 네트워크 상태에 의존하지 않는다. 실제 gateway · Ollama · Naver · PgVector 연결은 .http smoke로 분리해 필요할 때만 실행한다.

SECTION 08

08

트랜잭셔널 아웃박스 — messaging

도메인 쓰기와 outbox_events insert를 같은 트랜잭션 경계에 넣어 "커밋된 변경은 발행 의도도 함께 남는다"는 불변식을 만든다.



outbox_events 는 기능 테이블과 FK 를 맺지 않는다 — aggregate 참조는 DB 무결성이 아니라 이벤트 계약으로 해석한다.

구성 요소

DomainEventRecorder	core에 있는 기록 계약. 도메인 트랜잭션 안에서 호출한다
OutboxWriter	구현체. MANDATORY 전파로 호출자의 트랜잭션 참여를 강제한다
OutboxRelayService	트랜잭션 밖에서 claim → publish → 상태·다음 재시도 시각 기록

정직하게 말하면

- **at-least-once** — relay는 같은 이벤트를 두 번 보낼 수 있으므로 consumer는 event_id로 역등해야 한다. 버그가 아니라 계약이다.
- **relay는 기본 비활성**(relay-enabled=false)이고 외부 MQ는 확장 지점이다. 이 브랜치 기준 실행작은 "도메인 변경과 발행 의도를 한 트랜잭션에 원자적으로 남기는 것"까지다.
- 따라서 정확한 표현은 "메시지 큐를 운영한다"가 아니라 "**dual-write를 구조적으로 막았다**"이다.

얻는 것

- 커밋된 도메인 변경의 발행 의도가 반드시 DB에 남는다.
- broker와 relay가 느려도 도메인 트랜잭션이 네트워크 I/O를 기다리지 않는다.
- 즉시 exactly-once는 보장하지 않으며, 그 대가로 consumer 역등성을 요구한다.

SECTION 09

09

데이터 설계

하나의 테이블은 하나의 owning module만 갖는다. 다른 모듈이 남의 테이블을 직접 읽거나 쓰지 않는 것이 모듈러 모놀리스를 나중에 분리 가능하게 유지하는 최소 조건이다.

소유권 규칙

- owning module만 해당 테이블의 entity · repository · write policy를 변경한다.
- 다른 모듈이 필요하면 core port, 공개 application API, 또는 message contract로 우회한다.
- table · column · index · constraint를 바꾸면 같은 변경에서 ownership 문서를 갱신한다.
- destructive change는 PR 또는 별도 SQL artifact에 rollback / compatibility note를 남긴다.

관계형 테이블 소유권

Owner	Table	설명
auth	accounts	계정 identity, OAuth provider identity, optimistic lock version
auth	account_roles	account role set — accounts lifecycle에 종속
board	posts	본문, summary/tags/category/board_category/publish_origin, 조회수, like count, 작성자 nickname snapshot
board	post_tags	게시글 tag collection
board	comments	댓글/대댓글 tree. 작성자는 account_id scalar로 보관
board	post_like / comment_like	uniqueness (post_id, account_id) / (comment_id, account_id)
board	post_purchase_vote	출시 뉴스 구매 판단 투표. uniqueness (post_id, account_id)
board	post_file	S3 object metadata와 게시글 첨부 관계
board	post_product_links	게시글과 catalog product id의 느슨한 연결
board	post_reference_links	출시 뉴스 출처 evidence, canonical URL duplicate anchor, daily-cap metadata
ingest	tracked_keywords	스케줄 수집 대상 키워드와 source / display policy
ingest	collection_jobs	상품 수집 실행 단위 상태
ingest	raw_products	collection job별 외부 API 응답 원본 payload
catalog	product_categories / products / offers	카테고리 tree, 정규화 상품 truth, source-mall별 판매 상품 식별자
catalog	product_embeddings	product matching용 pgvector table · HNSW cosine index
catalog	product_match_candidates	자동 매칭 확신이 낮은 상품 병합 후보

Owner	Table	설명
price	price_snapshots	offer 수집 시점별 가격 스냅샷
messaging	outbox_events	standalone reliable event handoff table · 도메인 FK 없음
ai	vector_store	RAG document embeddings (Spring AI PgVector)

Redis 키와 S3 소유권

Owner	Key / Resource	용도와 실패 정책
auth	refresh_token:*	refresh token 상태 · fail-closed
auth	email_verify_token:* · email_verified:*	이메일 인증 token / 상태 · fail-closed
auth	email_verify_send:cooldown rate lock:email:*	이메일 발송 cooldown · rate · lock guard
auth	oauth2_code:*	짧은 TTL의 1회성 OAuth exchange code · fail-closed
auth	auth:login:rate fail lock:*	로그인 abuse guard
board	post:views:* · post:viewed:* · post:views:dirty	조회수 캐시 · 중복 방지 · 동기화 큐 · degraded
board	like:cooldown:* · like:rate:*	좋아요 abuse guard · fail-closed(429)
board	S3 bucket objects	post_file은 metadata만, object lifecycle은 board file 도메인 소유

PgVector 테이블을 둘로 나눈 이유

RAG 문서 검색용 vector_store는 ai가, 상품 매칭용 product_embeddings는 catalog가 소유한다. 두 테이블은 PostgreSQL의 vector 확장을 공유하지만 schema · index · dimension · embedding 입력의 소유권은 분리한다. ingest는 현재 monolith 단계에서 문서를 제출할 수 있을 뿐 PgVector schema 결정권은 없다. 나중에 서비스를 분리할 때 문서 쓰기 orchestration을 AI/RAG API나 message contract 뒤로 옮기기 위한 준비다.

Migration 규칙

런타임 마이그레이션은 app/src/main/resources/db/migration/의 VNNNN__description.sql에 둔다. 신규 빈 DB는 V0000 baseline부터 실행하고, production-like 환경은 Flyway 적용 후 Hibernate validate로 entity와 schema의 불일치를 잡는다. 로컬 개발은 기존 DB를 자동 편입하지 않기 위해 Flyway를 기본 비활성화한다. 각 migration은 primary owner 하나를 갖고 다음 header를 남긴다.

```
-- Owner:           -- Purpose:           -- Tables:
-- Compatibility: -- Rollback:           -- Verification:
```

SECTION 10

API 표면

10

애플리케이션 API는 /api/... 경로만 사용한다. Spring Security와 OAuth2 프레임워크가 소유하는 /oauth2/**, /login/oauth2/**와 Swagger · static 경로는 controller 명세에서 제외한다. 실행 중에는 Swagger UI가 현재 OpenAPI 스키마를 보여준다.

인증 표기와 공통 응답

표기	의미	공통 타입	필드
Public	인증 없이 호출 가능	PageResponse<T>	content, page, size, totalElements, totalPages, first, last, empty
Optional JWT	인증 없이 가능하되 JWT가 있으면 사용자별 상태 반영	MessageResponse	message
USER	ROLE_USER 필요	UrlResponse	url
Owner/ADMIN	리소스 소유자 또는 ROLE_ADMIN	ErrorResponse	status, error, message, validation?, timestamp
ADMIN	ROLE_ADMIN 필요		

엔드포인트

모듈	Method	Path	Auth	설명
auth	POST	/api/auth/register	Public	회원가입
auth	POST	/api/auth/login	Public	로그인 — access + refresh 발급
auth	POST	/api/auth/logout	USER	Redis refresh token 삭제
auth	POST	/api/auth/token/reissue	Public	refresh rotation 재발급
auth	POST	/api/auth/oauth2/exchange	Public	1회성 exchange code → token
auth	POST	/api/auth/email/send	Public	인증 메일 발송 (cooldown · rate · lock)
auth	GET	/api/auth/email/verify	Public	이메일 인증 확인
auth	GET/PATCH	/api/user/account · /nickname · /password	USER	계정 조회와 변경
board	GET	/api/posts	Optional JWT	목록 · 검색 · 필터 (batch 파생 조회)
board	POST	/api/posts	USER	작성 — category는 GENERAL 고정
board	GET	/api/posts/{postId}	Optional JWT	상세 — 인증 사용자만 조회수 증가
board	PUT/DELETE	/api/posts/{postId}	Owner/ADMIN	수정 · 삭제

모듈	Method	Path	Auth	설명
board	POST/DELETE	/api/posts/{postId}/like	USER	좋아요 토글 (guard + 먹등)
board	PUT/DELETE	/api/posts/{postId}/purchase-vote	USER	구매 판단 투표 (출시 뉴스 한정)
board	GET/POST	/api/posts/{postId}/comments	Optional JWT / USER	댓글 목록 · 작성
board	PUT/DELETE	/api/posts/{postId}/comments/{commentId}	Owner/ADMIN	댓글 수정 · 삭제
board	POST/DELETE	/api/posts/{postId}/comments/{commentId}/like	USER	댓글 좋아요
board	GET	/api/files/presigned-url · /{fileId}/download-url	USER	S3 presigned 업로드 · 다운로드
board	GET/PATCH	/api/user/profile · /nickname · /password	USER	프로필
board	GET	/api/products/{productId}/posts	Public	상품에 연결된 게시글
catalog	GET	/api/products · /search · /{productId}	Public	상품 목록 · 검색 · 상세
catalog	GET	/api/products/categories · /{categoryId}	Public	카테고리
catalog	POST/PATCH	/api/admin/products · /{productId}	ADMIN	상품 upsert
catalog	GET/PATCH	/api/admin/product-match-candidates · /approve · /reject	ADMIN	매칭 후보 승인 · 거절
price	GET	/api/products/{productId}/prices	Public	가격 이력
price	POST	/api/price-checks	USER	응답 전용 가격 판정 (현재 503)
ai	POST	/api/ai/chat	USER	RAG 채팅
ai	POST	/api/ai/generate	USER	게시글 초안 — 저장하지 않음
ingest	POST	/api/ingest/documents	USER	문서 chunk + embedding 적재
ingest	POST/GET/ PATCH	/api/admin/tracked-keywords ...	ADMIN	수집 키워드 등록 · 조회 · 비활성
ingest	POST/GET	/api/admin/collection-jobs/manual · /collection-jobs	ADMIN	수동 수집 실행 · 이력
ingest	POST	/api/admin/news-documents/manual	ADMIN	뉴스 문서 수집
ingest	POST	/api/admin/launch-news/manual	ADMIN	출시 뉴스 수동 발행

기본값은 열림이 아니라 닫힘

PostForgeAuthorizationRules의 마지막 규칙은 `anyRequest().denyAll()`이다. 규칙에 등록되지 않은 새 경로는 자동으로 공개되지 않는다.

SECTION 11

11

성능 엔지니어링

초기 부하 테스트에서 GET /posts가 느렸다. 하지만 "느리다"는 관찰만으로는 원인을 알 수 없다. 한 번에 한 변수만 분리하는 것이 이 장의 전부다.

접근 — 한 번에 한 변수만

- 1 **CPU 포화부터 제거** — 로그인과 읽기를 한 시나리오에 섞으면 1vCPU 서버가 BCrypt 해싱으로 포화돼 읽기 API의 진짜 병목을 가린다. 읽기 25 VU + 로그인 5 VU로 분리했다.
- 2 **SQL count 확인** — CPU가 여유로운데도 목록만 느리면 쿼리 수를 본다. Hibernate SQL 로그에서 요청 1건당 추가 쿼리 11개(N+1)를 확인했다.
- 3 **fetch 전략 교정** — 연관 로딩 전략과 batch size 조정.
- 4 **캐시 실험은 분리해서 판단** — 조회수 Redis 버퍼를 상세와 목록에 각각 A/B.

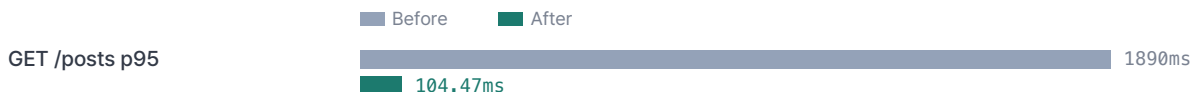
① 시나리오 분리 — CPU 포화 제거

시나리오	GET /posts med	GET /posts p95	시스템 신호
로그인 혼합 30 VU	498 ms	1.89 s	CPU 100%, Load 12.0
읽기 25 VU + 로그인 5 VU	90 ms	508 ms	CPU max 59%, Load 0.7

시나리오를 분리하자 CPU 포화는 해소됐지만 GET /posts는 /posts/{id}(p95 189~205 ms)보다 계속 느렸다. CPU가 여유 있는 상태에서 특정 조회 API만 느리면 다음으로 볼 것은 쿼리 수와 fetch 전략이다.

② N+1 제거와 batch size

단계	checks	전체 p95	GET /posts p95	GET /posts/{id} p95	로그인 p95
N+1 해결 후	6042/6042	285.04 ms	195.83 ms	191.04 ms	1.40 s
batch size 설정 후	7416/7416	180.75 ms	104.47 ms	97.98 ms	864.34 ms

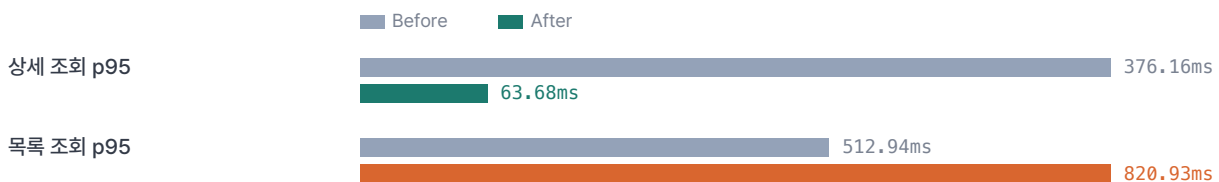


로그인 혼합 시나리오의 1.89 s에서 시나리오 분리 · N+1 제거 · batch size 조정을 거쳐 104.47 ms까지 내렸다.

③ 조회수 Redis 캐시 A/B — 같은 최적화가 API마다 반대로 작동했다

post-read.js / post-list.js, warmup 10 VU 10s → load 50 VU 30s → spike 100 VU 10s, 최대 150 VU. 에러율은 전후 모두 0.00%.

API	지표	Before	After	변화
게시글 상세	총 요청 수	9,918	20,347	2.05배
게시글 상세	평균	163.89 ms	26.08 ms	84.1% 감소
게시글 상세	p95	376.16 ms	63.68 ms	83.1% 감소
게시글 상세	RPS	198.36	406.94	2.05배
게시글 목록	총 요청 수	8,548	5,987	30.0% 감소
게시글 목록	평균	200.87 ms	341.07 ms	69.8% 증가
게시글 목록	p95	512.94 ms	820.93 ms	60.0% 증가
게시글 목록	RPS	170.96	119.74	30.0% 감소



여기서 배운 것

- 같은 최적화가 API마다 반대로 작동한다. 조회수 버퍼는 update가 잦은 상세 조회에는 크게 이로웠지만, 목록 조회는 파생 데이터를 더 읽고 리소스를 경쟁하면서 오히려 나빠졌다. 목록 최적화는 조회수 버퍼링 결정과 분리해서 다뤄야 한다.
- 수치는 방법이지 자량이 아니다. 위 값은 Oracle Cloud ARM 1vCPU / 1GB의 historical baseline이며 현재 운영(Intel N100) capacity 주장이 아니다. 값의 가치는 "20 RPS를 낸다"가 아니라 "병목을 재현 가능하게 분리하고 개선을 근거로 남겼다"는 절차에 있다.

외부 연동 계측

HTTP latency만으로는 튜닝 대상을 고를 수 없다. 외부 fetch · DB persist · LLM generation을 앱 내부 metric으로 분리해 남긴다.

Metric	용도
external_source_fetch_seconds	Naver 등 외부 source API 호출 시간
external_source_db_persist_seconds	source 결과를 DB / vector store에 반영하는 시간
ai_text_generation_seconds	LLM 생성 호출 시간
ai_text_generation_prompt_tokens	LLM input token 처리량 계산
ai_text_generation_completion_tokens	LLM output TPS 계산

SECTION 12

설계 결정 기록 (ADR)

12

구조 설명이 아니라 왜 그렇게 골랐는지, 무엇을 검토했다가 버렸는지를 남긴다. 세 편 모두 현재 코드에 반영되어 있다.

ADR-001 · 조회수에 Redis 사용

배경	상세 조회는 자주 호출되고, 매 요청마다 posts.views를 즉시 update하면 write amplification과 row contention이 생긴다. 같은 사용자의 반복 조회는 24시간에 한 번만 카운트해야 한다.
결정	조회수는 Redis를 write buffer로 쓰고 account 단위 24시간 중복을 막는다. 변경된 post만 스케줄러가 RDB로 반영한다. 좋아 요는 Redis write-behind로 처리하지 않는다 .
영향	읽기 요청의 DB update 빈도가 줄고 Redis increment로 빠르게 응답한다. dirty set만 처리하므로 keyspace가 커져도 sync 범위가 제한된다. Redis 재시작 시 마지막 sync 이후 증가분은 유실될 수 있고, 허용 손실 범위는 스케줄러 주기에 묶인다.

검토했지만 선택하지 않은 대안	이유
RDB 즉시 update	조회 트래픽이 증가할수록 write contention이 커진다
Redis KEYS post:views:* scan	운영 Redis에서 O(N) 블로킹 위험이 있다
좋아요까지 Redis write-behind	사용자별 좋아요 상태는 정합성 요구가 높아 DB source of truth가 더 안전하다

ADR-002 · Refresh Token Rotation

배경	access token을 stateless JWT로 처리하면 로그아웃 · 장기 세션 유지 · 탈취 토큰 무효화 같은 상태성 요구를 다루기 어렵다.
결정	refresh token을 계정별 Redis key refresh_token:{accountId}에 저장하고 재발급마다 rotation한다. 요청 token은 저장값과 상수 시간 비교로 검증하고, 통과하면 계정 활성 상태와 role을 다시 조회해 새 token 쌍을 발급한다.
영향	로그아웃과 비밀번호 변경은 Redis key 삭제로 다음 재발급을 차단한다. 탈취된 refresh token은 rotation 이후 저장값과 불일치 해 재사용이 막힌다. 다만 이미 발급된 access token은 stateless라 만료 전까지 유효하다 — 즉시 폐기가 필요하면 blacklist · token version · logout-after timestamp 같은 추가 상태가 필요하다.
후속	강한 single-use rotation이 필요해지면 Redis Lua script나 compare-and-set 계약으로 validate + replace를 원자화한다.

ADR-003 · 모듈러 모놀리스 (Accepted)

현재 선택은 "MSA를 바로 도입하지 않고, MSA 후보 경계를 코드 안에서 먼저 검증하는 modular monolith"다. 하나의 process로 실행해 배포 · 트랜잭션 · 로컬 디버깅 · 운영 관측 비용을 낮춘다.

MSA를 먼저 도입하지 않는 이유

- 서비스 간 네트워크 실패 · timeout · retry · circuit breaker 설계가 즉시 필요해진다.

- 분산 트랜잭션을 피하기 위한 event · outbox · idempotency 설계가 필요해진다.
- 서비스별 observability · 배포 파이프라인 · secret · ingress · 인증 정책을 따로 운영해야 한다.
- 아직 검증되지 않은 도메인 경계를 API로 고정하면 이후 변경 비용이 커진다.

아직 해결하지 않은 것

현재 구조는 MSA가 아니다. 분리 시점에는 서비스별 DB ownership과 schema migration 전략, 트랜잭션 경계를 넘는 흐름의 event / outbox / idempotency 설계, API contract versioning과 backward compatibility 정책, 서비스별 observability · deployment · secret · ingress · rate limit이 추가로 필요하다. 미해결 항목은 migration 도구 도입이 아니라 **서비스별 DB로 분리할 때의 migration · 호환성 전략**이다.

SECTION 13

트레이드오프와 한계

13

이 장은 약점 고백이 아니라 설계 판단의 일부다. 무엇을 알면서 남겨뒀고, 왜 지금은 그래도 되는지, 확장하면 어떻게 푸는지를 함께 적는다.

1 Refresh 토큰 검증과 교체가 원자적이지 않다

한계	refresh 재발급은 Redis 저장값과 요청 토큰을 상수 시간 비교로 검증한 뒤 새 토큰으로 교체한다. 이 validate + replace가 단일 원자 연산이 아니라서, 같은 refresh 토큰으로 동시 재발급이 들어오면 둘 다 검증을 통과하고 마지막 저장분만 살아남을 수 있다.
지금 괜찮은 이유	정상 클라이언트는 재발급을 순차로 하므로 창이 매우 좁고, rotation 자체는 탈취 토큰 재사용을 여전히 차단한다.
확장 시 해법	Redis Lua script나 compare-and-set 저장 계약으로 원자화한다. 클라이언트는 중복 재발급 중 하나가 INVALID_TOKEN으로 실패하는 흐름을 정상으로 처리한다.

2 아웃박스 relay가 기본 비활성이다

한계	도메인 변경과 발행 의도를 한 트랜잭션에 남기는 구조까지가 실동작이고, 실제 relay와 외부 MQ 연동은 커서 붙이는 확장 지점이다.
지금 괜찮은 이유	현재 제품 흐름은 동기 처리로 충분하고, dual-write를 막는 핵심(커밋과 발행 의도의 원자성)은 이미 확보돼 있다.
확장 시 해법	relay-enabled=true로 켜고 MQ adapter를 붙인다. at-least-once이므로 consumer를 event_id 기준으로 먹등하게 만든다.

3 Redis 실패 정책이 용도별로 다르다 (의도된 판단)

한계	조회수는 Redis 재시작 시 마지막 동기화 이후 변경분을 최대 5분까지 잃을 수 있다.
지금 괜찮은 이유	이건 실수가 아니라 용도별 정책이다. 조회수는 정확도보다 가용성이 중요한 파생 데이터라 degraded로 두고, 인증(refresh · OAuth2 · 이메일 인증)의 Redis는 fail-closed로 두어 상태를 잃느니 요청을 실패시킨다.
확장 시 해법	동기화 주기를 줄이거나 write-ahead 로그로 유실 창을 좁힐 수 있다. 단, 조회수에 그만한 비용을 쓸 가치가 있는지부터 판단한다.

4 성능 수치는 historical baseline이다

한계	문서의 20~25 RPS와 p95 개선회는 모두 Oracle Cloud ARM 1vCPU / 1GB에서 측정한 과거 값이다. 현재 운영(Intel N100)의 새 capacity baseline은 아직 없다.
지금 괜찮은 이유	애초에 capacity를 과장하지 않으려고 historical / current를 명시적으로 구분해 기록했다. 병목 분석의 방법론과 상대 개선회는 환경이 바뀌어도 유효하다.
확장 시 해법	N100에서 같은 k6 시나리오를 돌리고, 같은 시간 구간의 CPU · memory · GC · DB · Redis 지표를 함께 수집해 새 baseline을 남긴다.

5 목록 조회는 조회수 캐시로 오히려 느려졌다

한계	조회수 Redis 버퍼는 상세 조회를 크게 개선했지만 목록 조회 p95는 약 60% 악화됐다.
지금 괜찮은 이유	A/B로 회귀를 정량 확인했고, 목록 최적화를 조회수 버퍼링과 분리해야 한다는 결론을 근거와 함께 남겼다.
확장 시 해법	목록은 전용 projection DTO · 커버링 인덱스 · fetch 전략으로 따로 최적화한다. 조회수 버퍼는 상세에만 이롭다는 전제를 유지한다.

6 가격 판정 외부 소스가 현재 비활성이다

한계	Naver Shopping 검색 API 종료 대응으로 가격 판정에 사용할 실시간 외부 샘플이 없다.
지금 괜찮은 이유	가짜 데이터나 종료된 API를 호출하지 않고 503으로 가용성 부족을 명시한다. 판정 결과도 저장하거나 게시글로 만들지 않는다.
확장 시 해법	대체 상품 소스를 SourceRequestExecutor에 연결하고 기존 판정 계산을 재사용한다.

7 게시글·댓글은 아직 물리 삭제다

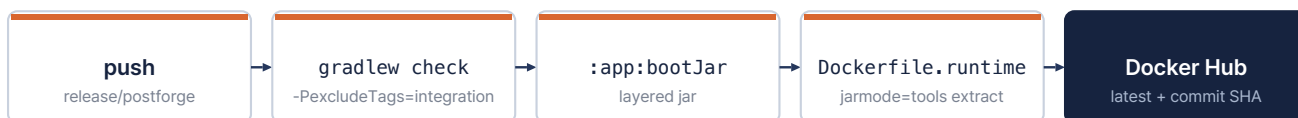
한계	현재 구현은 게시글/댓글을 hard delete한다. 복구나 감사 이력이 남지 않는다.
지금 괜찮은 이유	MVP 범위에서 삭제 정책의 기준은 "일반 사용자에게 더 이상 노출하지 않는 것"이고, 계정은 이미 상태 기반 soft delete를 target으로 잡아뒀다.
확장 시 해법	게시글/댓글도 상태 기반 soft delete로 전환하고, 물리 삭제는 별도 보존 기간과 정리 작업으로 옮긴다.

SECTION 14

운영과 배포

14

release/postforge 브랜치의 CI가 테스트 성공 후 Spring Boot layered jar 기반 runtime image를 만들고 Docker Hub에 latest와 commit SHA 태그로 게시한다.



runtime image 설계

- dependencies / spring-boot-loader / snapshot-dependencies / application 레이어를 분리해 registry cache 적중률을 높인다.
- non-root 사용자(postforge)로 실행하고 -XX:MaxRAMPercentage=75.0 -XX:+ExitOnOutOfMemoryError 를 기본 옵션으로 둔다.

CI 구성

트리거	release/postforge · refactor/postforge-ddd push, release/postforge · master PR. docs/**와 README 변경은 paths-ignore로 제외
test job	JDK 21(temurin) → wrapper validation → ./gradlew check -PexcludeTags=integration. 실패 시 test report를 artifact로 업로드
image job	test 성공 + release/postforge push일 때만 실행. bootJar → buildx → Docker Hub push
동시성	workflow + ref 단위 concurrency group으로 이전 실행을 취소해 러너를 낭비하지 않는다

관측

Micrometer + Prometheus로 metric을 노출하고 Grafana에서 본다. actuator 경로는 ActuatorSecurityConfig로 따로 보호한다. JVM · HTTP 기본 지표 외에 외부 source fetch, DB persist, LLM generation을 별도 timer로 계측해 "느린 게 우리 코드인지 외부인지"를 분리한다. 로그는 ECS JSON 형식으로 구조화한다.

환경변수 그룹

APP_CORS_ALLOWED_ORIGINS APP_OAUTH2_REDIRECT_URL APP_EMAIL_VERIFICATION_BASE_URL
 POSTGRES_* DB_POOL_MAX_SIZE REDIS_HOST/PORT LLM_PROVIDER LLM_CHAT_BASE_URL
 LLM_CHAT_MODEL LLM_EMBEDDING_* GOOGLE_CLIENT_* NAVER_CLIENT_* KAKAO_CLIENT_*
 GMAIL_* AWS_REGION/BUCKET JWT_SECRET MONITORING_* NAVER_API_HUB_*
 NAVER_NEWS_ENABLED

I 테스트 실행

전체 테스트 / 통합 테스트 제외 / 실행 jar 생성

```
./gradlew test
./gradlew test -PexcludeTags=integration
./gradlew :app:bootJar -PexcludeTags=integration
```

전용 부하 테스트 runner는 현재 저장소에 없다. 과거 k6 · Bruno · Grafana · API smoke 산출물은 docs/performance/에 historical evidence로 보관하고, 학습 자료는 docs/learning/으로 분리해 성능 근거로 쓰지 않는다.

SECTION 15

문서 체계와 마무리

15

같은 표나 규칙을 여러 문서에 복사하지 않고, 소유 문서에 상세를 두고 나머지는 링크한다. 코드 · 설정 · 엔드포인트와 어긋난 설명을 발견하면 정본부터 고친다.

위치	정본 책임
docs/api/	endpoint, DTO, HTTP status, 인증 조건
docs/policy/	권한, 데이터 소유, 비용, 삭제, retention invariant
docs/architecture/	runtime 흐름, 모듈 경계, infrastructure ownership
docs/architecture/flows/	요청부터 저장소까지의 내부 처리 흐름
docs/database/	schema ownership, ERD, DBML, Flyway 규칙
docs/decisions/	선택 이유, 대안, 트레이드오프
docs/performance/	측정 방법, 결과, historical baseline
docs/troubleshooting/	증상, 확인 순서, 복구, 재발 방지
docs/portfolio/	리뷰어를 위한 요약과 근거 탐색 순서
docs/learning/	학습 · 설계 참고자료 — 현재 동작이나 성능 증거로 사용하지 않음

정리

PostForge에서 설명할 가치가 있는 것은 기능의 목록이 아니라 세 가지 절차다.

#	주제	한 줄
H1	성능 병목의 체계적 분리	추측 대신 CPU / SQL count / p95로 병목을 분리해 GET /posts p95를 1.89 s → 104 ms로 개선. 캐시가 오히려 악화된 케이스도 근거와 함께 기록
H2	트랜잭셔널 아웃박스	도메인 쓰기와 이벤트 발행 의도를 같은 트랜잭션에 남겨 dual-write를 제거. MANDATORY 전파로 원자성을 강제
H3	모듈 경계와 포트 계약	허용된 application API와 core 포트로 결합 방향을 제한하고 ArchUnit으로 검증

이 문서에 대하여

저장소의 README, docs/architecture, docs/database, docs/decisions, docs/performance, docs/portfolio와 실제 소스 코드(controller · security rule · build.gradle · CI workflow · Dockerfile)를 근거로 작성했다. 성능 수치는 모두 Oracle Cloud ARM 1vCPU / 1GB 환경의 historical baseline이며 현재 운영 환경의 capacity 주장이 아니다.